

```

1 // STA mode but accepts connections //
2 // static ip //
3 // Port forwarding //
4
5 #include <WiFi.h>
6
7 // choose a port number as mentioned in the text //
8 #define EspListenPort xxxxx
9
10 // Replace with your router credentials
11 const char* ssid = "....."; //
12 const char* password = ".....";
13
14 WiFiServer server( EspListenPort ); // STA mode //
15
16 // Set your Static IP address for ESP. See text //
17 // Octets are separated by comma and not by dot //
18 IPAddress localIP( xxx, xxx, xxx, xxx );
19 IPAddress gateway( xxx, xxx, xxx, xxx );
20 IPAddress subnet( xxx, xxx, xxx, xxx );
21

```

Fig. 3: Wi-Fi credentials and static IP address configuration

```

45 delay(500);
46 Serial.print(".");
47 }
48 // Print local IP address and start web server
49 Serial.println("");
50 Serial.println("WiFi connected.");
51 Serial.print ("IP address: ");
52 Serial.println(WiFi.localIP());
53 server.begin();
54 }
55
56 void loop() {
57 // ESP32 Listens for incoming clients //
58 // ESP in STA mode but listens //
59 client = server.available();
60
61 if (client) { // If a new client connects,
62
63 while (client.connected()) {
64
65 if (client.available()) {
66 ch = client.read();
67 Serial.write(ch);
68 switch (ch) {
69 case '1':
70 client.println ( "ON" );
71 break;
72 case '2':
73 client.println ( "OFF" );
74 break;
75 }
76 }
77
78 switch ( ch ) {
79 case '1':
80 digitalWrite(ledPin, HIGH);
81 break ;
82
83 case '2':
84 digitalWrite(ledPin, LOW);
85 break ;
86 }
87 }
88 }
89 delay (500);
90 }
91 //.....end of code.....//
92

```

Fig. 4: Source code on Arduino platform

```

// NetLink Configuration : needs only ESP32 IP and Port
// Port Forwarding

#include <windows.h>
#include <stdio.h>

#pragma comment(lib,"ws2_32.lib")
LRESULT CALLBACK WndProc(HWND, UINT, WPARAM, LPARAM);
int CreateButtEdit(HINSTANCE, HWND);
int FromFilePortIp();
int LapToEspThr();
DWORD WINAPI LapToEsp(LPVOID lpParam);
char *trim(char *s);
int SavePortIpFile();

#define INIT_SOCK WM_USER+100
#define SaveId 1
#define CloId 2
#define EdMessId 3
#define But1Id 4
#define But2Id 5
#define ConId 7
#define EdIpId 8
#define EdPortId 9

char PortIpNam[] = "PortAndIP.txt"; // Port no and IP //

char EspIPAddr[20] = "";
u_short EspLPort;

HINSTANCE hInstance;
HWND hwnd;
HANDLE hButSave, hButCon, hEdMess;
HANDLE hEdIp, hEdPort;
HANDLE hBut1, hBut2;
HANDLE hLapThr = NULL;
SOCKET LapSock = INVALID_SOCKET;

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance,
PSTR szCmdLine, int iCmdShow)
{
static TCHAR szAppName[] = TEXT("ESP32");
MSG msg;
WNDCLASS wndclass;

wndclass.style = CS_HREDRAW | CS_VREDRAW;
wndclass.lpfnWndProc = WndProc;

```

Fig. 5: Screenshot of the C program

Then install ESP32 by Espressif Systems via the Board Manager. This completes the setup.

ESP32 code. The internet operates on a client-server model. The ESP32 operates in Station (STA) mode but functions as a server, listening for incoming connections.

To assign a static IP address to the ESP32, examine the IP addresses assigned by the Wi-Fi router to other devices and choose an unused IP by modifying the last octet. A listening port number also needs to be defined for the ESP32.

A port number in the range of 32767 to 65535 is recommended.

The code can be written and uploaded using the Arduino IDE. The Wi-Fi SSID and password should be configured in the code based on the local network. The static IP address must then be assigned in the code. Fig. 3 shows the configuration of Wi-Fi credentials and the static IP address. Fig. 4 shows the source code written using the Arduino platform.

In the relevant part of the code, the ESP32 checks the status received from the client to determine whether to turn the LED on or off.

Under the Tools menu in the Arduino IDE, the appropriate ESP32 board and COM port must be selected. Once selected, the code can be compiled and uploaded. If necessary, the Boot button on the ESP32 can be held down while uploading. Fig. 5 shows the output screenshot.

Client-side code. The client-side code is developed in Visual Studio 2015, and a user interface (UI) is created to control the device. Fig. 5 shows a screenshot of the C program. Fig. 6 shows the UI layout.

The port number on which the ESP32 is listening must be entered. The public IP address of the router should also be entered and saved using the Save button. (The public